

DEPARTMENT OF COMPUTER APPLICATIONS

Mini-Project MMC202-Object Oriented Programming using Java

“SECURE LOGIN SYSTEM WITH EMAIL OTP(2MFA)”



Submitted by

Uday C K

4AD25MC086

Rohith Gowda S

4AD25MC064

Under the Guidance of

Ms. Sushmitha B H

Assistant Professor

ATMECE, Mysore

DEPARTMENT OF COMPUTER APPLICATIONS

EVALUATION SHEET

SI. NO	NAME	USN	PRESENTATION (20)	VIVA (10)	REPORT (20)	TOTAL (50)
1	UDAY C K	4AD25MC086				
2	ROHITH GOWDA S	4AD25MC064				

DEPARTMENT OF COMPUTER APPLICATIONS

SECURE LOGIN SYSTEM

A **Secure Login System** is a software application designed to authenticate users and protect sensitive information by allowing access only to authorized individuals. It is an essential security component used in modern web, desktop, and mobile applications to prevent unauthorized access and safeguard user data.

The system verifies the identity of users through a secure authentication process. During registration, users create an account by providing a unique username, email address, and password. Instead of storing passwords in plain text, the system uses **BCrypt hashing** to securely encrypt passwords before storing them in the database.

To provide an additional layer of security, the system implements **One-Time Password (OTP) authentication**. After the user enters the correct username and password, an OTP is generated and sent to the registered email address using an SMTP email service. The user must enter the correct OTP within a specified time limit to complete the login process. This two-step verification significantly reduces the risk of unauthorised access.

The system incorporates several security features, including input validation, password hashing using BCrypt, OTP verification, prepared statements to prevent SQL injection, session management, and audit logging. Together, these features provide strong protection against common security threats such as brute-force attacks, password theft, SQL injection, and unauthorised access.

In conclusion, a Secure Login System provides a reliable and secure authentication mechanism by combining encrypted password storage, OTP-based verification, audit logging, and secure database practices. It enhances user trust, protects confidential information, and ensures that only authenticated users gain access to system resources

DEPARTMENT OF COMPUTER APPLICATIONS

Abstract

Traditional login systems that rely solely on a username and a static password are increasingly vulnerable to credential theft, phishing, brute-force attacks, and password reuse across multiple platforms. This project presents the design and implementation of a Two-Factor Authentication (2FA) Login System that strengthens the conventional login process by introducing a second, independent verification step. After a user successfully supplies a valid username and password, a six-digit One-Time Password (OTP) is generated and delivered to the user's registered email address; access is granted only after this OTP is correctly verified within a limited time window.

The system is built using Java 17 and Spring Boot 3.2.12 for the backend REST API, MySQL 8 for persistent storage, and the Jakarta Mail API for SMTP-based email delivery. Passwords and OTPs are never stored in plain text; both are hashed using BCrypt (jBCrypt) before being persisted. The application is offered in two operating modes: a web application with an HTML/CSS/JavaScript front end communicating with the Spring Boot REST API on port 8081, and a menu-driven console application for demonstration and testing without a browser. Every authentication-related action, including login attempts, OTP verification, and password changes, is recorded in an audit log table to support traceability and security review.

The key outcomes of this project include a fully working two-step authentication flow with OTP expiry (5 minutes), single-use enforcement, and a maximum of three verification attempts; a normalized three-table MySQL schema (users, otp_records, login_audit); comprehensive unit and integration testing using JUnit 5 and Mockito; and a security posture that defends against common threats such as SQL injection, password guessing, and OTP replay.

The project demonstrates how a relatively lightweight set of technologies can be combined to deliver a authentication mechanism comparable in principle to those used by major real-world services such as online banking portals and webmail providers.

DEPARTMENT OF COMPUTER APPLICATIONS

Table of Contents

Sl. No.	Contents	Page No.
1	Abstract	1
2	Chapter 1 – Introduction	2
	2.1 Background on Login Security	2
	2.2 What is Two-Factor Authentication (2FA)	2
	2.3 Why Email-Based OTP is Practical	3
	2.4 Problem Statement	3
	2.5 Objectives of the Project	4
3	Chapter 2 – Literature Survey / Related Work	5
	3.1 Existing Login Methods and Their Weaknesses	5
	3.2 How OTP Improves Security	5
	3.3 Real-World Examples	6
4	Chapter 3 – System Analysis	7
	4.1 Functional Requirements	7
	4.2 Non-Functional Requirements	7
	4.3 Software Requirements	8
	4.4 Hardware Requirements	8
5	Chapter 4 – Technologies Used	9
	5.1 Java 17	9
	5.2 Spring Boot 3.2.12	9
	5.4 Jakarta Mail API	10

DEPARTMENT OF COMPUTER APPLICATIONS

SL. No.	Contents	Page No
	5.5 BCrypt (jBCrypt)	10
	5.6 Maven	10
	5.7 JUnit 5 and Mockito	11
	5.8 HTML/CSS/JavaScript	11
6	Chapter 5 – System Design	12
	6.1 Overall Architecture	12
	6.2 Database Design	13
	6.3 Entity Relationship (ER) Diagram	14
	6.4 Class Diagram Overview	15
7	Chapter 6 – Modules Description	16
	7.1 User Registration	16
	7.2 Login – Password Verification	17
	7.3 OTP Issuance	17
	7.4 OTP Verification	17
	7.5 Resend OTP	18

DEPARTMENT OF COMPUTER APPLICATIONS

SLNO.	Contents	Page No.
	7.6 Change Password	18
	7.7 Logout	19
	7.8 Audit Logging	19
8	Chapter 7 – Implementation Details	20
	8.1 Key Files	20
	8.2 Key Code Logic Explained	20
	8.3 REST API Endpoints	21
9	Chapter 8 – Security Features	23
	9.1 BCrypt Hashing	23
	9.2 OTP Expiry & Attempt Limits	23
	9.3 SQL Injection Prevention	23
	9.4 Audit Logging	23
	9.5 Configuration File	23
	9.6 Password Strength Validation	24
	9.7 Session Management	24
10	Chapter 9 – Testing	25
	10.1 Testing Tools	25
	10.2 Test Categories	25

DEPARTMENT OF COMPUTER APPLICATIONS

SL.No	Contents	Page No.
	10.4 Running the Test Suite	26
	10.5 Importance of Unit Testing	26
11	Chapter 10 – Output / Screenshots	27
12	Chapter 11 – Future Enhancements	29
13	Chapter 12 – Conclusion	30
	13.1 Summary of What Was Achieved	30
	13.2 Why the System is Secure	30
	13.3 Lessons Learned	30
14	Chapter 13 – References	31

DEPARTMENT OF COMPUTER APPLICATIONS

Abstract

Traditional login systems that rely solely on a username and a static password are increasingly vulnerable to credential theft, phishing, brute-force attacks, and password reuse across multiple platforms. This project presents the design and implementation of a Two-Factor Authentication (2FA) Login System that strengthens the conventional login process by introducing a second, independent verification step. After a user successfully supplies a valid username and password, a six-digit One-Time Password (OTP) is generated and delivered to the user's registered email address; access is granted only after this OTP is correctly verified within a limited time window.

The system is built using Java 17 and Spring Boot 3.2.12 for the backend REST API, MySQL 8 for persistent storage, and the Jakarta Mail API for SMTP-based email delivery. Passwords and OTPs are never stored in plain text; both are hashed using BCrypt (jBCrypt) before being persisted. The application is offered in two operating modes: a web application with an HTML/CSS/JavaScript front end communicating with the Spring Boot REST API on port 8081, and a menu-driven console application for demonstration and testing without a browser. Every authentication-related action, including login attempts, OTP verification, and password changes, is recorded in an audit log table to support traceability and security review.

The key outcomes of this project include a fully working two-step authentication flow with OTP expiry (5 minutes), single-use enforcement, and a maximum of three verification attempts; a normalized three-table MySQL schema (users, otp_records, login_audit); comprehensive unit and integration testing using JUnit 5 and Mockito; and a security posture that defends against common threats such as SQL injection, password guessing, and OTP replay.

The project demonstrates how a relatively lightweight set of technologies can be combined to deliver a authentication mechanism comparable in principle to those used by major real-world services such as online banking portals and webmail providers.

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 1

Introduction

1.1 Background on Login Security

User authentication is the process of confirming that a person requesting access to a system is indeed who they claim to be. For decades, the dominant model of authentication on the web and in enterprise software has been the single-factor username-and-password combination. While simple to implement and easy for users to understand, this model suffers from well-documented weaknesses. Passwords are frequently reused across services, are chosen to be memorable rather than strong, and can be captured through phishing emails, keyloggers, database breaches, or simple guessing. Once an attacker obtains a valid username and password pair, nothing else stands between them and the protected account.

As data breaches have become common and password databases are routinely leaked or sold on underground markets, relying on a password alone is no longer considered adequate protection for anything beyond the least sensitive systems. Security researchers and industry bodies increasingly recommend that any system handling personal, financial, or otherwise sensitive data should require at least one additional, independent factor of verification alongside the password.

1.2 What is Two-Factor Authentication (2FA)

Two-Factor Authentication (2FA) is a security mechanism that requires a user to present two distinct forms of evidence, drawn from two different categories of authentication factors, before being granted access. The three broadly recognized categories are: something the user knows (a password or PIN), something the user has (a mobile phone, hardware token, or registered email account), and something the user is (a biometric trait such as a fingerprint or facial pattern). By combining a knowledge factor with a possession factor, 2FA ensures that even if an attacker learns a user's password, they cannot complete the login process unless they also control the second factor.

DEPARTMENT OF COMPUTER APPLICATIONS

This project implements 2FA using a knowledge factor (the password) combined with a possession factor realized through the user's registered email inbox. After the password is verified, the system generates a short-lived, randomly generated numeric OTP and emails it to the address on file. The user must retrieve this code from their inbox and enter it into the application within a limited time frame to complete the login.

1.3 Why Email-Based OTP is Practical

Several possession factors could theoretically be used for the second authentication step, including SMS text messages, dedicated authenticator apps (such as Google Authenticator or Authy using Time-based One-Time Passwords), hardware security keys (such as YubiKeys using FIDO2/WebAuthn), and email delivery. Email-based OTP was selected for this project for a combination of practical and academic reasons.

- **Universality:** virtually every user already possesses and regularly checks an email account, whereas SMS requires a registered mobile number and telecom gateway integration, and hardware keys require the purchase and distribution of physical devices.
- **Low infrastructure cost:** sending OTPs by email requires only an SMTP server and the Jakarta Mail API, both of which are free or low-cost to set up for an academic project, unlike SMS gateways which are typically metered and paid per message.
- **Sufficient security uplift for the target use case:** while SMS and authenticator apps offer certain advantages (e.g., resistance to email account compromise), email OTP still meaningfully raises the bar over password-only authentication by requiring control of a second, independent account.

1.4 Problem Statement

Conventional single-factor login systems used in many small and medium-scale applications provide no protection once a password is compromised, whether through phishing, credential-stuffing attacks that reuse leaked password lists, or simple weak password choices. There is a need for a login system that: (a) verifies the user's identity using more than one independent factor,

DEPARTMENT OF COMPUTER APPLICATIONS

(b) protects stored credentials against exposure even if the underlying database is compromised, (c) limits the window of opportunity and number of attempts available to an attacker attempting to bypass the second factor, and (d) maintains a verifiable audit trail of all authentication events for accountability and post-incident analysis. This project addresses that need by designing and implementing a complete two-factor, email-OTP-based login system with hashed credential storage, time-boxed and attempt-limited OTP verification, and comprehensive audit logging.

1.5 Objectives of the Project

- To design and implement a secure user registration and login workflow that separates password verification (step one) from OTP verification (step two).
- To generate and deliver time-limited, single-use, six-digit numeric OTPs to a user's registered email address using the Jakarta Mail API over SMTP.
- To store all passwords and OTPs using one-way BCrypt hashing so that plaintext credentials are never persisted in the database.
- To enforce OTP expiry (5 minutes), single-use consumption, and a maximum of three verification attempts before an OTP is invalidated.
- To provide two access modes: a Spring Boot REST API with an HTML/CSS/JavaScript front end, and a menu-driven console application, so that the same core authentication logic can be exercised both through a browser and from the command line.
- To maintain a complete, queryable audit log of registration, login, OTP, and password-change events using a dedicated login_audit table.
- To validate the correctness and robustness of the system through automated unit and integration tests written with JUnit 5 and Mockito.

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 2

Literature Survey / Related Work

2.1 Existing Login Methods and Their Weaknesses

A review of commonly deployed authentication mechanisms reveals a spectrum of trade-offs between usability and security, summarized below.

Method	Description	Key Weakness
Plain username/password	Single knowledge factor compared against a stored value	Vulnerable to phishing, credential stuffing, brute force, and database leaks if passwords are not hashed
Password + security question	Adds a secondary knowledge factor (e.g., mother's maiden name)	Answers are often guessable, publicly discoverable, or reused, providing weak additional protection
CAPTCHA-protected login	Adds a challenge to distinguish humans from bots	Prevents automated abuse but does nothing to stop a human attacker with a stolen password
SMS-based OTP	Sends a numeric code to the user's mobile number	Vulnerable to SIM-swap attacks, requires paid SMS gateway integration, dependent on network coverage
Authenticator app TOTP	Time-based codes generated locally on a registered device	Requires app installation and device provisioning; loss of device can lock the user out
Email-based OTP	Sends a numeric code to the user's registered email inbox	Depends on email account security, but requires no special hardware and is simple to deploy

2.2 How OTP Improves Security

A One-Time Password fundamentally changes the security calculus of a login attempt because it introduces a credential that is valid for a single use and for a short, bounded period of time.

DEPARTMENT OF COMPUTER APPLICATIONS

Even if an attacker successfully phishes or brute-forces a user's static password, they cannot complete authentication without also gaining real-time access to the second factor. Because the OTP is generated fresh for every login attempt and expires quickly, it cannot be harvested in advance and replayed later, unlike a static password. Limiting the number of verification attempts (three, in this project) further defeats automated guessing of the six-digit code, since the search space of 1,000,000 possible codes cannot be brute-forced within three tries.

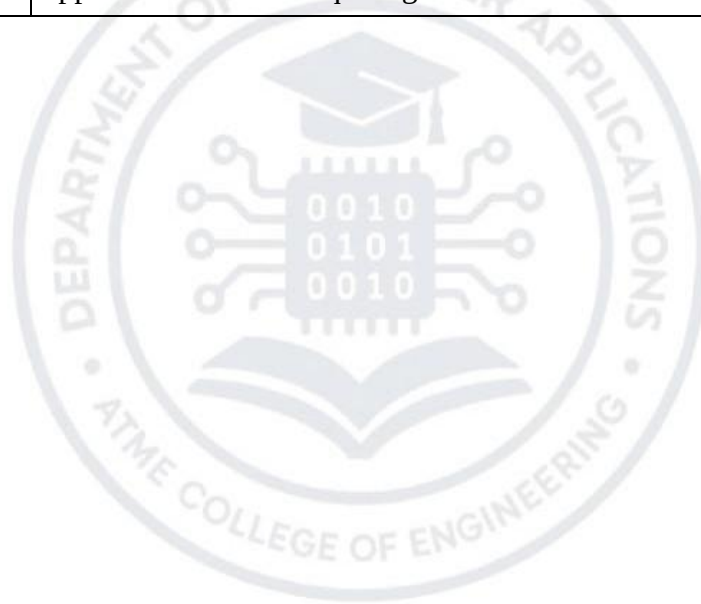
Academic and industry literature on authentication consistently finds that layering an independent possession factor on top of a knowledge factor substantially reduces the success rate of account takeover attacks, even when the underlying password has already been compromised through an unrelated breach. This is the central justification for adopting a 2FA approach in this project rather than continuing to rely on password-only authentication.

2.3 Real-World Examples: Banking, Gmail, and Beyond

- Online banking portals in India and worldwide almost universally require an OTP, delivered by SMS or email, before authorizing login or high-value transactions, reflecting regulatory requirements for strong customer authentication in the financial sector.
- Gmail and other major webmail providers offer 2-Step Verification, allowing users to receive a verification code via SMS, an authenticator app, or a prompt on a trusted device after the password step succeeds.
- E-commerce platforms frequently use OTP verification during account recovery, first-time login from a new device, or high-risk transactions such as changing a registered email or shipping address.
- Enterprise Single Sign-On (SSO) solutions such as Okta and Microsoft Entra ID (Azure AD) commonly enforce 2FA/MFA policies for corporate accounts, especially for remote access to internal systems.

DEPARTMENT OF COMPUTER APPLICATIONS

Usability	The web front end must provide clear, immediate feedback for invalid input, incorrect passwords, expired OTPs, and successful actions.
Performance	OTP generation and email dispatch should complete within a few seconds under normal network conditions to avoid degrading user experience.
Reliability	The system should gracefully handle SMTP delivery failures and present the user with a clear resend option rather than crashing.
Maintainability	The codebase should follow a layered architecture (Controller-Service-DAO) to keep concerns separated and testable.
Portability	The console application should provide the same core functionality as the web application without requiring a browser.



DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 3

System Analysis

3.1 Functional Requirements

- FR1 - Register: A new user shall be able to create an account by supplying a unique username, a valid email address, and a password that satisfies the system's strength policy.
- FR2 - Login (Password Step): A registered user shall be able to submit their username and password; the system shall verify the password against the stored BCrypt hash.
- FR3 - OTP Issuance: Upon successful password verification, the system shall generate a six-digit numeric OTP, hash and store it with an expiry timestamp, and email it to the user's registered address.
- FR4 - OTP Verification: The user shall submit the OTP received by email; the system shall verify it against the stored hash, checking expiry and attempt count before granting access.
- FR5 - Resend OTP: The user shall be able to request a new OTP if the original expires or is not received, subject to a cool-down period to prevent abuse.
- FR6 - Change Password: An authenticated user shall be able to change their password, which shall itself be re-hashed with BCrypt before storage.
- FR7 - Logout: An authenticated user shall be able to terminate their session/token, ending the ability to access protected resources.
- FR8 - Audit Logging: The system shall record every registration, login attempt, OTP event, and password change with a timestamp, username, action, and result (success/failure) in the login_audit table.

3.2 Non-Functional Requirements

Category	Requirement
Security	Passwords and OTPs must never be stored or logged in plaintext; all secrets are hashed using BCrypt with an appropriate work factor.
Security	OTPs must expire after 5 minutes and be usable only once; verification must be limited to 3 attempts per issued OTP.

DEPARTMENT OF COMPUTER APPLICATIONS

Usability	The web front end must provide clear, immediate feedback for invalid input, incorrect passwords, expired OTPs, and successful actions.
Performance	OTP generation and email dispatch should complete within a few seconds under normal network conditions to avoid degrading user experience.
Reliability	The system should gracefully handle SMTP delivery failures and present the user with a clear resend option rather than crashing.
Maintainability	The codebase should follow a layered architecture (Controller-Service-DAO) to keep concerns separated and testable.
Portability	The console application should provide the same core functionality as the web application without requiring a browser.

3 Software Requirements

- Java Development Kit (JDK) 17 or later
- Spring Boot 3.2.12 (Spring Web, Spring Data JPA / JDBC)
- MySQL Server 8.x
- Apache Maven (build and dependency management)
- Jakarta Mail API with an SMTP provider (e.g., Gmail SMTP or an equivalent mail relay)
- jBCrypt library for password/OTP hashing
- JUnit 5 and Mockito for automated testing
- A modern web browser (Chrome, Firefox, or Edge) for the web front end
- IDE such as IntelliJ IDEA or Eclipse for development

3.4 Hardware Requirements

Component	Minimum Specification
Processor	Dual-core 2.0 GHz or equivalent
RAM	4 GB (8 GB recommended for smooth IDE + MySQL + Maven builds)
Storage	1 GB free disk space for project, dependencies, and database
Network	Internet connectivity for SMTP-based OTP email delivery

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 4

Technologies Used

4.1 Java 17

What it is: Java 17 is a long-term support (LTS) release of the Java programming language and platform, offering modern language features such as records, sealed classes, and pattern matching for switch, alongside the mature Java Virtual Machine (JVM) ecosystem.

Why it was chosen: Java was chosen for its strong typing, wide industry adoption in enterprise backend systems, mature security libraries, and excellent compatibility with the Spring Boot framework.

Role in the project: Java 17 is the implementation language for the entire backend: the REST API, the console application, the service and DAO layers, and the utility classes for password hashing and OTP generation.

4.2 Spring Boot 3.2.12

What it is: Spring Boot is an opinionated framework built on top of the Spring ecosystem that simplifies the creation of stand-alone, production-ready Java applications, providing auto-configuration, embedded servers, and a large set of starter dependencies.

Why it was chosen: Spring Boot was selected to minimize boilerplate configuration, provide a clean way to expose REST endpoints, and integrate easily with Spring Data for database access and Spring Security concepts for authentication.

Role in the project: Spring Boot powers the web application mode of the project, exposing REST endpoints for registration, login, OTP verification, resend, password change, and logout, and wiring together the Controller, Service, and DAO layers.

4.3 MySQL 8

DEPARTMENT OF COMPUTER APPLICATIONS

What it is: MySQL is a widely used open-source relational database management system (RDBMS) that stores data in structured tables and supports SQL for querying and manipulation.

Why it was chosen: MySQL was chosen for its reliability, ACID-compliant transactions, wide tooling support, and familiarity within academic and industry settings, making it well suited for storing structured, relational authentication data.

Role in the project: MySQL hosts the three core tables of the system: users, otp_records, and login_audit, and is accessed through the DAO layer using JDBC/Prepared Statements.

4.4 Jakarta Mail API

What it is: The Jakarta Mail API (formerly JavaMail) is a Java library that provides a platform-independent framework for sending and receiving email using protocols such as SMTP, IMAP, and POP3.

Why it was chosen: It was chosen because it directly supports SMTP-based email dispatch, which is the mechanism used to deliver the OTP to the user's registered email address, without requiring a third-party paid gateway.

Role in the project: The EmailService class in this project uses Jakarta Mail to connect to an SMTP server and send the OTP email during the login flow and whenever a resend is requested.

4.5 BCrypt (jBCrypt)

What it is: BCrypt is a password-hashing function designed to be computationally expensive and to incorporate a per-hash random salt, making brute-force and rainbow-table attacks impractical. jBCrypt is the Java library implementing this algorithm.

Why it was chosen: BCrypt was chosen over faster general-purpose hash functions (such as MD5 or SHA-256 alone) because its deliberate slowness and built-in salting specifically defend against offline password-cracking attempts.

DEPARTMENT OF COMPUTER APPLICATIONS

4.6 Maven

What it is: Apache Maven is a build automation and dependency management tool for Java projects, using a declarative pom.xml file to describe dependencies, plugins, and build lifecycle phases.

Why it was chosen: Maven was chosen for its widespread adoption, straightforward dependency resolution for Spring Boot, Jakarta Mail, jBCrypt, JUnit, and Mockito, and its simple mvn test / mvn package workflow.

Role in the project: Maven manages the project's dependencies and build lifecycle, including compiling source code, running the JUnit/Mockito test suite via mvn test, and packaging the application as an executable JAR.

4.7 JUnit 5 and Mockito

What it is: JUnit 5 is the standard framework for writing and running unit and integration tests in Java, while Mockito is a mocking framework that allows dependencies to be simulated in isolation during testing.

Why it was chosen: These were chosen because they are the de-facto industry standard for Java testing, allowing service-layer logic (such as OTP validation) to be tested without needing a live database or SMTP server.

Role in the project: JUnit 5 and Mockito are used to test password hashing, OTP generation and expiry logic, input validation, service-layer business rules, and controller endpoint behaviour.

4.8 HTML/CSS/JavaScript

What it is: HTML, CSS, and JavaScript are the foundational technologies of the web front end, used respectively for page structure, visual styling, and client-side interactivity.

Why it was chosen: A plain HTML/CSS/JS front end was chosen (rather than a heavier framework such as React or Angular) to keep the client lightweight and focused on directly calling the Spring Boot REST endpoints for demonstration purposes.

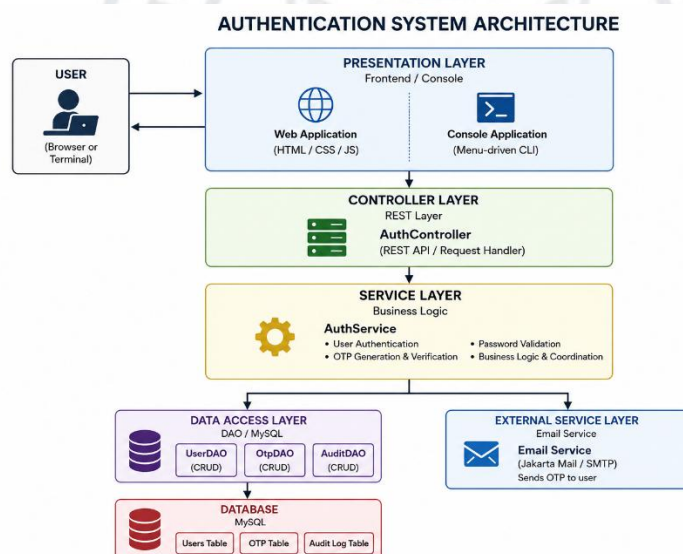
DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 5

System Design

5.1 Overall Architecture

The system follows a layered (n-tier) architecture that cleanly separates presentation, control, business logic, data access, and external communication concerns. This separation improves testability (each layer can be tested or mocked independently), maintainability (a change in the database schema, for example, is largely confined to the DAO layer), and reusability (the same Service layer is invoked by both the REST controller for the web application and the console menu handler).



Layer-by-layer Explanation

User Layer: Represents the human actor interacting with the system, either through a web browser (web application mode) or a terminal session (console application mode).

Frontend / Console Layer: In web mode, this is the HTML/CSS/JavaScript client that renders forms and calls REST endpoints using `fetch()`. In console mode, this is a menu-driven command-line interface that prompts the user for input and displays results directly in the terminal.

DEPARTMENT OF COMPUTER APPLICATIONS

Controller Layer: AuthController exposes REST endpoints (in web mode) that receive HTTP requests, perform basic request validation, and delegate all business logic to the Service layer. It contains no direct database or email logic itself.

Service Layer: AuthService contains the core business rules of the system, including verifying passwords against stored hashes, generating and validating OTPs, enforcing expiry and attempt limits, and orchestrating calls to the DAO and Email layers.

DAO / Database Layer: UserDao, OtpDao, and an audit DAO encapsulate all SQL interactions with MySQL using PreparedStatement queries, keeping raw SQL out of the Service layer and reducing the risk of SQL injection.

Email Service Layer: EmailService wraps the Jakarta Mail API to compose and send the OTP email over SMTP, isolating email-specific configuration (host, port, credentials) from the rest of the application.

5.2 Database Design

The relational schema consists of three tables designed to satisfy third normal form (3NF), avoiding redundant storage of user information across tables and using foreign keys to maintain referential integrity between users and their associated OTP and audit records.

users Table

Column	Type	Constraints
user_id	BIGINT	Primary Key, Auto Increment
username	VARCHAR(50)	Unique, Not Null
email	VARCHAR(100)	Unique, Not Null
password_hash	VARCHAR(60)	Not Null (BCrypt hash)
created_at	TIMESTAMP	Default CURRENT_TIMESTAMP

DEPARTMENT OF COMPUTER APPLICATIONS

otp_records Table

Column	Type	Constraints
otp_id	BIGINT	Primary Key, Auto Increment
user_id	BIGINT	Foreign Key -> users(user_id)
otp_hash	VARCHAR(60)	Not Null (BCrypt hash of OTP)
expires_at	TIMESTAMP	Not Null (issued time + 5 minutes)
attempts	INT	Default 0, Max 3
used	BOOLEAN	Default FALSE

login_audit Table

Column	Type	Constraints
audit_id	BIGINT	Primary Key, Auto Increment
user_id	BIGINT	Foreign Key -> users(user_id), Nullable
action	VARCHAR(30)	e.g., REGISTER, LOGIN, OTP_SENT, OTP_VERIFY, PASSWORD_CHANGE, LOGOUT
status	VARCHAR(10)	SUCCESS or FAILURE
timestamp	TIMESTAMP	Default CURRENT_TIMESTAMP

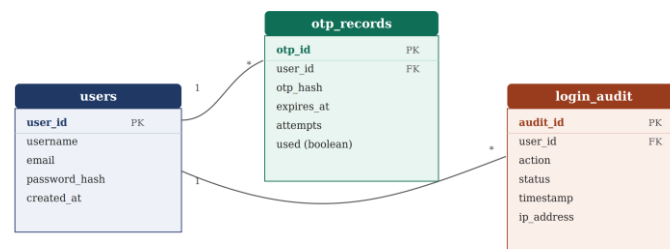
5.3 Entity-Relationship (ER) Diagram Description

The users entity is the central entity of the schema, related to otp_records in a one-to-many relationship, since a single user may have multiple OTP records generated over time (one per login attempt or resend request), though only the most recent unexpired, unused record is considered valid at any moment. The otp_records and login_audit tables do not have a direct relationship with each other; they are both linked back to users through the shared user_id foreign key.

DEPARTMENT OF COMPUTER APPLICATIONS

3 Entity-Relationship (ER) Diagram Description

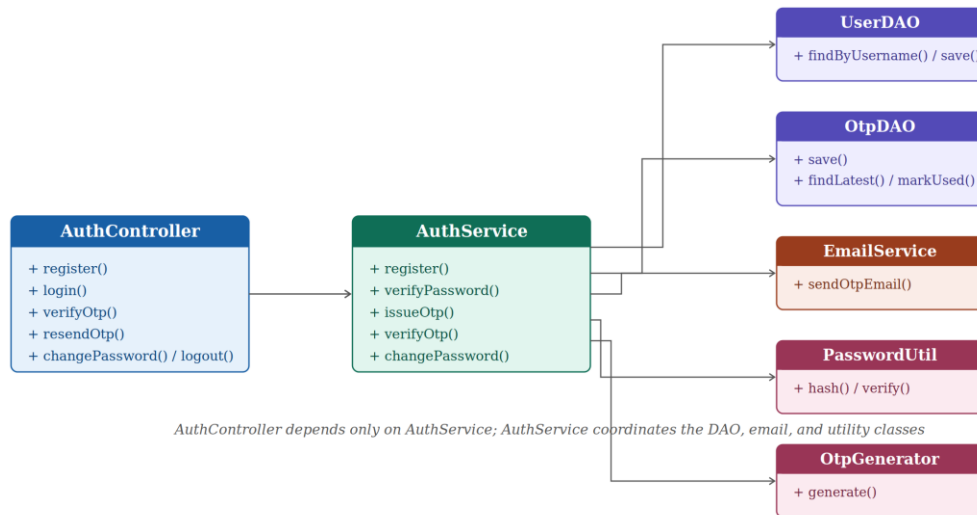
The users entity is the central entity of the schema, related to otp_records in a one-to-many relationship, since a single user may have multiple OTP records generated over time (one per login attempt or resend request), though only the most recent unexpired, unused record is considered valid at any moment. Similarly, users has a one-to-many relationship with login_audit, since every authentication-related action performed by (or attempted for) a user produces a new audit row. The otp_records and login_audit tables do not have a direct relationship with each other; they are both linked back to users through the shared user_id foreign key.



Both otp_records and login_audit reference users through the shared user_id foreign key (one user, many OTP and audit rows)

5.4 Class Diagram Overview

At the application layer, the design follows a clear separation of responsibility between five categories of classes: Controllers (handling HTTP requests), Services (business logic), DAOs (persistence), Email/Utility classes (cross-cutting concerns), and Generators (OTP creation). The relationships are illustrated below.



AuthController depends only on AuthService, never directly on the DAO or Email layers, preserving the layered architecture. AuthService, in turn, coordinates UserDAO and OtpDAO for persistence, EmailService for OTP delivery, PasswordUtil for hashing/verification of both

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 6

Modules Description

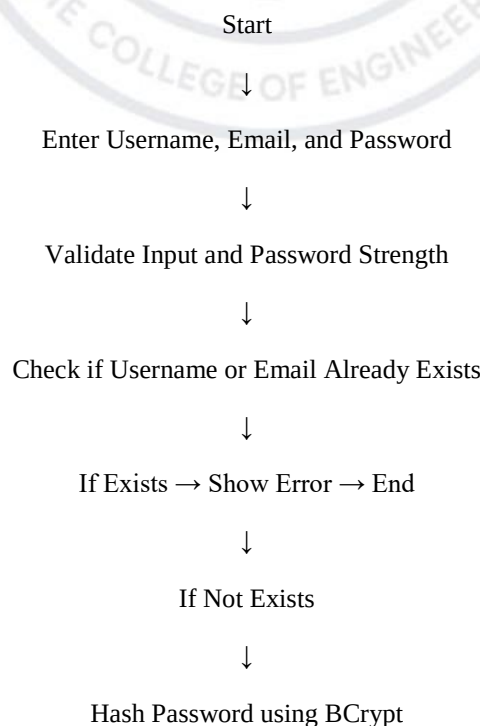
6.1 Module 1: User Registration

Purpose: Allows a new user to create an account by supplying a username, email address, and password, which is validated and stored securely for future logins.

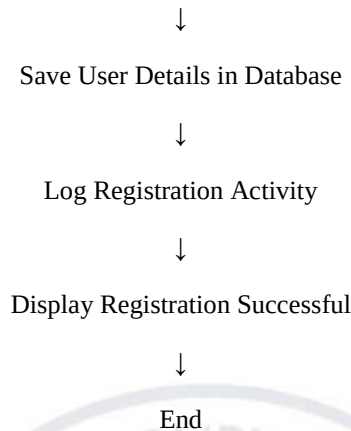
Input / Output: Input: username, email, password (plaintext, over HTTPS). Output: success confirmation with new user_id, or a validation error (duplicate username/email, weak password).

Flow Explanation: The Controller receives the registration request and forwards it to AuthService, which checks that the username and email are not already registered, validates password strength, hashes the password using BCrypt via PasswordUtil, and passes the new record to UserDao for insertion. A REGISTER event is written to login_audit.

Work flow:



DEPARTMENT OF COMPUTER APPLICATIONS



6.2 Module 2: Login - Step 1 (Password Verification)

Purpose: Verifies that the submitted username and password match a registered account before proceeding to the OTP stage.

Input / Output: Input: username, password. Output: 'password verified, OTP sent' status, or 'invalid credentials' error.

Flow Explanation: AuthController passes the credentials to AuthService, which retrieves the stored password hash via UserDao and compares it against the submitted password using BCrypt's verify function. A successful match triggers OTP issuance (Module 3); a failure logs a LOGIN FAILURE event and returns a generic error message that does not reveal whether the username or password was incorrect, to avoid aiding attackers.

6.3 Module 3: OTP Issuance - Step 2

Purpose: Generates a fresh six-digit OTP after successful password verification and delivers it to the user's registered email address.

Input / Output: Input: user_id (from Module 2). Output: OTP emailed to user; a hashed OTP record stored with expiry timestamp; UI transitions to OTP-entry screen.

Flow Explanation: AuthService invokes OtpGenerator to produce a random six-digit numeric code, hashes it with PasswordUtil/BCrypt, and stores it via OtpDAO together with an

DEPARTMENT OF COMPUTER APPLICATIONS

expires_at timestamp set to 5 minutes from issuance and attempts initialized to 0. EmailService then composes and sends the OTP email using Jakarta Mail over SMTP. An OTP_SENT audit event is logged.

6.4 Module 4: OTP Verification - Step 3

Purpose: Validates the OTP entered by the user against the stored hashed OTP, enforcing expiry and attempt-count rules before granting full access.

Input / Output: Input: username (or session token), OTP entered by the user. Output: authentication success (session/JWT issued) or failure (invalid, expired, or attempts exhausted).

Flow Explanation: AuthService retrieves the latest unused OTP record for the user via OtpDAO, checks whether the current time exceeds expires_at (rejecting if expired), checks whether attempts already equal 3 (rejecting and requiring a resend if so), and otherwise verifies the entered OTP against the stored hash using BCrypt. On success, the record is marked used, a session/token is issued, and an OTP_VERIFY SUCCESS event is logged. On failure, attempts is incremented and an OTP_VERIFY FAILURE event is logged.

6.5 Module 5: Resend OTP

Purpose: Allows the user to request a new OTP if the original was not received or has expired, while preventing abuse through a cool-down restriction.

Input / Output: Input: username / session identifier. Output: a newly issued OTP emailed to the user, replacing the previous one, or a cool-down error if requested too soon.

Flow Explanation: AuthService checks the timestamp of the most recently issued OTP; if a minimum cool-down interval has not yet elapsed, the request is rejected to prevent email flooding. Otherwise, the previous OTP record is invalidated and Module 3 (OTP Issuance) is re-invoked to generate and send a new code.

6.6 Module 6: Change Password

Purpose: Enables an authenticated user to update their account password securely.

DEPARTMENT OF COMPUTER APPLICATIONS

Input / Output: Input: current password, new password (with confirmation). Output: success confirmation, or error if current password is incorrect or new password fails strength validation.

Flow Explanation: AuthService re-verifies the user's current password against the stored hash as a safeguard, validates the new password against the strength policy, hashes it with BCrypt, and updates the record via UserDao. A PASSWORD_CHANGE audit event is logged regardless of outcome.

6.7 Module 7: Logout

Purpose: Terminates the user's authenticated session, ensuring that protected resources can no longer be accessed without re-authentication.

Input / Output: Input: active session/token. Output: session invalidated confirmation.

Flow Explanation: The Controller receives the logout request, and AuthService invalidates the corresponding session or token (in web mode) or clears the in-memory authenticated state (in console mode). A LOGOUT event is recorded in login_audit.

6.8 Module 8: Audit Logging

Purpose: Provides a persistent, queryable record of every authentication-related action taken within the system, supporting traceability and security review.

Input / Output: Input: action type, associated user_id (if known), and result status, supplied internally by every other module. Output: a new row appended to the login_audit table.

Flow Explanation: Rather than being triggered directly by the user, this module is invoked internally by every other module (Registration, Login, OTP Issuance, OTP Verification, Resend, Change Password, Logout) each time a relevant action occurs, ensuring a complete and consistent trail of events for later inspection by an administrator.

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 7

Implementation Details

7.1 Key Files

The backend implementation is organised into a small set of focused classes, each with a single, well-defined responsibility, following standard Spring Boot project conventions.

File	Responsibility
AuthController.java	Exposes REST endpoints for registration, login, OTP verification, resend, password change, and logout; delegates all logic to AuthService.
AuthService.java	Contains the core business logic: password verification, OTP issuance/verification rules, expiry and attempt-limit enforcement, password change logic.
EmailService.java	Wraps the Jakarta Mail API to compose and send OTP emails over SMTP using configured host, port, and credentials.
UserDAO.java	Handles all SQL operations against the users table (insert, find by username/email, update password) using PreparedStatement.
OtpDAO.java	Handles all SQL operations against the otp_records table (insert, find latest valid OTP, increment attempts, mark used).
PasswordUtil.java	Provides static helper methods to hash and verify passwords and OTPs using BCrypt (jBCrypt).
OtpGenerator.java	Generates a cryptographically reasonable random six-digit numeric OTP using SecureRandom.

7.2 Key Code Logic Explained

7.2.1 Password Hashing and Verification

PasswordUtil exposes a hash(String plain) method that calls BCrypt.hashpw() with a randomly generated salt (BCrypt.gensalt()), producing a 60-character hash string that embeds the salt and work factor. A corresponding verify(String plain, String hash) method calls BCrypt.checkpw(), which internally re-derives the hash using the stored salt and performs a constant-time comparison, preventing timing attacks. This same utility is reused for both passwords and OTPs, ensuring consistent, well-tested hashing behaviour across the codebase.

DEPARTMENT OF COMPUTER APPLICATIONS

```
// Simplified logic (not full source)

String hash(String plain) {

    return BCrypt.hashpw(plain, BCrypt.gensalt(12));

}

boolean verify(String plain, String hash) {

    return BCrypt.checkpw(plain, hash);

}
```

7.2.2 OTP Generation

OtpGenerator uses `java.security.SecureRandom`, rather than the weaker `java.util.Random`, to produce a random integer in the range 100000-999999, guaranteeing a fixed six-digit format with no leading-zero ambiguity. `SecureRandom` is preferred here because it draws from a cryptographically strong source of entropy, making the generated OTP significantly harder to predict than one produced by a standard pseudo-random generator.

7.2.3 OTP Expiry and Attempt Enforcement

When an OTP record is created, its `expires_at` column is set to the issuance time plus five minutes. During verification, `AuthService` first compares the current server time against `expires_at`; if the OTP has expired, verification is rejected immediately without even checking the submitted code, and the user is prompted to request a resend. Separately, the attempts counter is checked against a hard limit of three; once reached, further verification attempts against that OTP record are rejected regardless of correctness, forcing the user to request a new OTP. This dual check (time-based and attempt-based) closes two different attack windows: a slow brute-force attempt across many attempts, and a delayed replay of a captured OTP after its intended validity window.

7.2.4 SQL Injection Prevention

All DAO classes exclusively use `PreparedStatement` with parameterized placeholders (?) rather than concatenating user input directly into SQL strings. This ensures that any characters submitted by a user, including SQL metacharacters such as quotes or semicolons, are treated strictly as literal data values by the MySQL driver and never as executable SQL syntax.

DEPARTMENT OF COMPUTER APPLICATIONS

7.3 REST API Endpoints

Method	Path	Description
POST	/api/auth/register	Registers a new user with username, email, and password.
POST	/api/auth/login	Step 1: verifies username/password and triggers OTP issuance on success.
POST	/api/auth/verify-otp	Step 2: verifies the submitted OTP and completes login.
POST	/api/auth/resend-otp	Issues a new OTP, subject to a cool-down restriction.
POST	/api/auth/change-password	Changes the password for an authenticated user.
POST	/api/auth/logout	Invalidates the current session/token.
GET	/api/auth/audit	(Admin/testing use) Retrieves recent entries from the login_audit table.

All endpoints accept and return JSON payloads and are designed to run behind HTTPS in a production deployment (HTTP is used locally for development on port 8081 for simplicity).

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 8

Security Features

8.1 BCrypt Hashing for Passwords and OTPs

Both passwords and OTPs are hashed using BCrypt before being written to the database. BCrypt's adaptive work factor and built-in per-value salting mean that even if the users or otp_records tables were exposed through a breach, an attacker could not feasibly recover the original plaintext values through brute force within any practical time frame, and identical passwords or OTPs chosen by different users would still produce different stored hashes.

8.2 OTP Expiry, Single-Use, and Attempt Limits

- Expiry: every OTP is valid for a maximum of 5 minutes from issuance, after which it is automatically rejected regardless of correctness.
- Single-use: once an OTP has been successfully verified, its record is marked used, and it cannot be verified a second time even if resubmitted.
- Attempt limits: a maximum of 3 verification attempts are permitted per issued OTP; exceeding this threshold forces the user to request a fresh OTP via the Resend module.

8.3 SQL Injection Prevention via PreparedStatement

Every database query issued by UserDao, OtpDao, and the audit DAO is constructed using PreparedStatement with bound parameters rather than string concatenation. This is the standard, industry-recommended defence against SQL injection, and ensures that malicious input supplied through the username, email, or any other field cannot alter the structure or intent of the executed SQL statement.

8.4 Audit Logging

Every registration, login attempt, OTP issuance, OTP verification, resend request, password change, and logout event is recorded in the login_audit table with a timestamp and success/failure status. This provides a durable, queryable trail that supports intrusion detection

DEPARTMENT OF COMPUTER APPLICATIONS

(e.g., spotting a burst of failed login attempts against a single account) and post-incident forensic analysis.

8.5 Configuration File for Credentials

Database connection credentials and SMTP mail server credentials are externalized into a configuration file (application.properties / application.yml, or environment variables in a production deployment) rather than being hardcoded into Java source files. This prevents secrets from being accidentally committed to version control and allows different credentials to be used across development, testing, and production environments without code changes.

8.6 Password Strength Validation

Newly chosen or updated passwords are validated against a minimum strength policy before being accepted, typically requiring a minimum length together with a mix of uppercase, lowercase, numeric, and special characters. This reduces the likelihood that a user selects a trivially guessable password, which would otherwise weaken the security contribution of the first authentication factor.

8.7 Session Management

Once both the password and OTP steps succeed, the system issues a session identifier or token that represents the authenticated state of the user for subsequent requests (such as viewing the dashboard or changing the password). This session/token is invalidated explicitly on logout and is designed to have a reasonable expiry to limit the impact of a session being left open on a shared or unattended device.

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 9

Testing

9.1 Testing Tools

Automated testing for this project is implemented using JUnit 5, the standard framework for writing and executing unit and integration tests in modern Java applications, together with Mockito, a mocking framework used to simulate dependencies such as the database DAOs and EmailService so that business logic in AuthService can be tested in isolation, without requiring a live MySQL instance or a real SMTP connection.

9.2 Test Categories

Category	What is Tested
Hashing	PasswordUtil.hash() produces distinct hashes for identical inputs (due to salting); verify() correctly accepts matching and rejects non-matching plaintext/hash pairs.
OTP Generation	OtpGenerator.generate() always produces a six-digit numeric string within the expected 100000-999999 range across repeated invocations.
Validation	Username/email uniqueness checks, password strength rules, and OTP format validation correctly accept valid input and reject invalid input.
Service Logic	AuthService correctly enforces OTP expiry, attempt-limit rejection, single-use consumption, and resend cool-down, using mocked OtpDAO and EmailService.
Controller	AuthController endpoints return correct HTTP status codes and JSON response bodies for both success and failure scenarios, using a mocked AuthService.

DEPARTMENT OF COMPUTER APPLICATIONS

9.3 Sample Test Scenarios

Test Case	Description	Expected Result
<code>verifyOtp_rejectsExpiredOtp()</code>	Verify that an expired OTP cannot be used for authentication.	OTP verification fails with the message " OTP expired. "
<code>verifyOtp_rejectsAfterThreeFailedAttempts()</code>	Verify that an OTP is rejected after three failed verification attempts.	OTP verification fails with the message " Maximum attempts reached. "
<code>hashAndVerify_roundTripSucceeds()</code>	Verify that a password hashed using BCrypt can be successfully verified with the original plaintext password.	Password verification returns true , confirming successful hashing and verification.

9.4 Running the Test Suite

The full automated test suite is executed with a single Maven command: `mvn test`

This command compiles the project, runs every JUnit 5 test class under `src/test/java`, and reports the number of tests run, passed, failed, and skipped, along with a summary of any assertion failures or exceptions encountered.

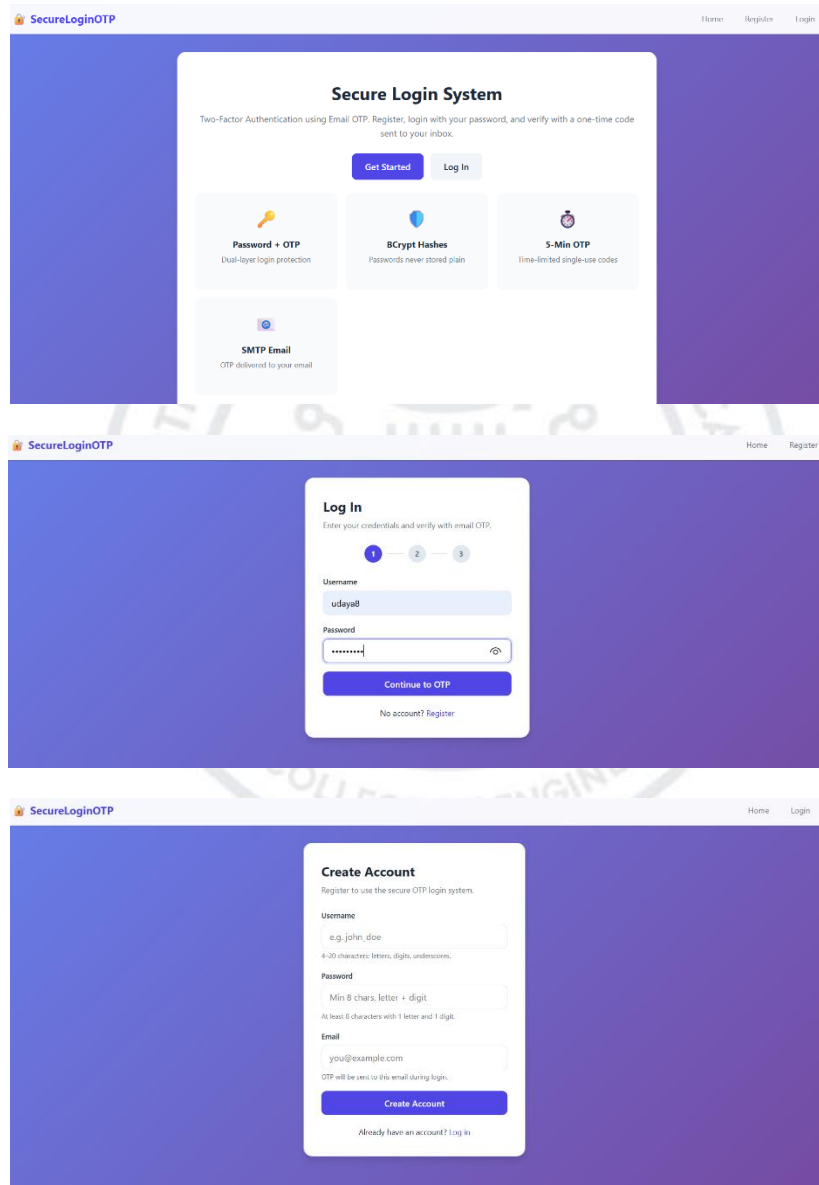
9.5 Importance of Unit Testing

Unit testing is especially important for a security-sensitive module such as authentication, because subtle logic errors (for example, an off-by-one error in the attempt counter, or an inverted expiry comparison) can silently weaken or entirely defeat the intended security guarantees without producing any obviously visible symptom during manual testing. By codifying the expected behaviour of expiry, attempt-limiting, single-use consumption, and hashing as automated tests, regressions introduced by future code changes can be caught immediately, and the correctness of the security-critical logic can be demonstrated repeatedly rather than relying solely on ad-hoc manual verification.

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 10

Output / Screenshots

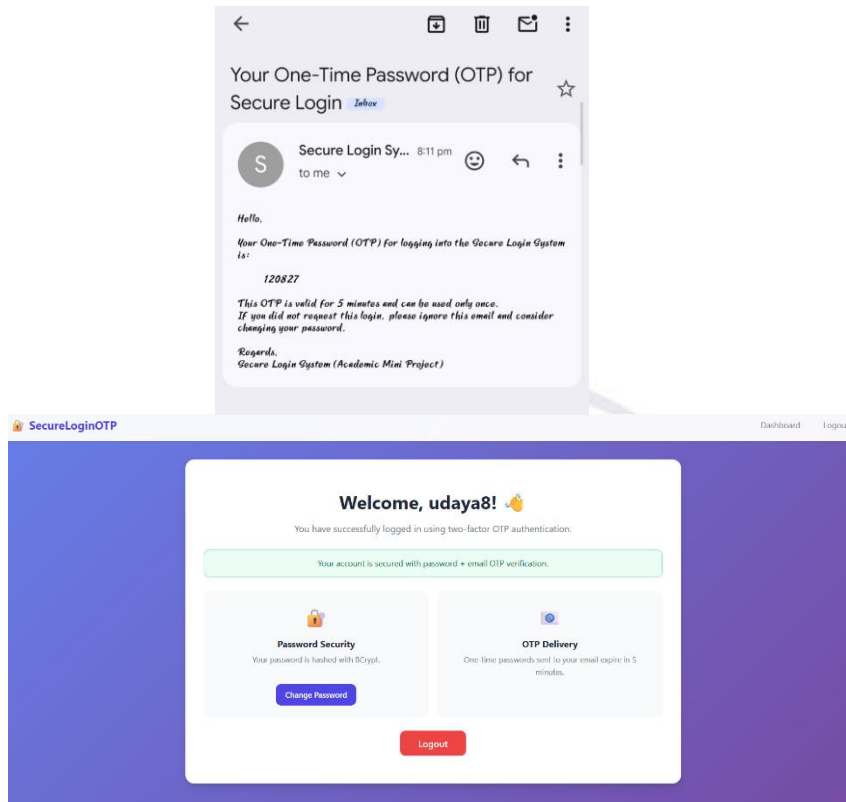


The first screenshot shows the 'Secure Login System' landing page. It features a header with 'SecureLoginOTP' and navigation links for 'Home', 'Register', and 'Login'. The main content area has a title 'Secure Login System' and a subtitle 'Two-Factor Authentication using Email OTP. Register, login with your password, and verify with a one-time code sent to your inbox.' Below this are two buttons: 'Get Started' and 'Log In'. There are four feature cards: 'Password + OTP' (Dual-layer login protection), 'BCrypt Hashes' (Passwords never stored plain), '5-Min OTP' (Time-limited single-use codes), and 'SMTP Email' (OTP delivered to your email).

The second screenshot shows the 'Log In' page. It has a header with 'SecureLoginOTP' and navigation links for 'Home' and 'Register'. The main content area has a title 'Log In' and a subtitle 'Enter your credentials and verify with email OTP.' Below this is a progress indicator with three steps: 1 (selected), 2, and 3. There are two input fields: 'Username' (with the value 'udaya8') and 'Password' (with masked characters). Below these is a 'Continue to OTP' button and a link 'No account? Register'.

The third screenshot shows the 'Create Account' page. It has a header with 'SecureLoginOTP' and navigation links for 'Home' and 'Login'. The main content area has a title 'Create Account' and a subtitle 'Register to use the secure OTP login system.' Below this are three input fields: 'Username' (with the value 'e.g. john_doe'), 'Password' (with the value 'Min 8 chars, letter + digit'), and 'Email' (with the value 'you@example.com'). Below these is a 'Create Account' button and a link 'Already have an account? Log In'.

DEPARTMENT OF COMPUTER APPLICATIONS



DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 11

Future Enhancements

- **Account Lockout:** temporarily lock an account after a configurable number of consecutive failed password or OTP attempts, adding a further layer of brute-force protection beyond the existing per-OTP attempt limit.
- **SMS OTP via Twilio:** extend the second factor to support SMS delivery through the Twilio API as an alternative or fallback channel to email, for users who prefer or require mobile-based verification.
- **React/Angular Frontend:** replace the current plain HTML/CSS/JavaScript client with a component-based single-page application framework such as React or Angular, improving maintainability and enabling richer client-side state management.
- **Email Verification at Registration:** require new users to confirm ownership of their registered email address via a verification link before the account becomes active, preventing registration with unowned or mistyped email addresses.
- **Password Reset Link:** introduce a self-service 'forgot password' flow using a time-limited, single-use reset link emailed to the user, complementing the existing authenticated change-password flow.
- **Docker Deployment:** containerize the Spring Boot application and MySQL database using Docker and Docker Compose, simplifying environment setup, deployment consistency, and portability across development and production machines.

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 12

Conclusion

12.1 Summary of What Was Achieved

This project successfully designed and implemented a complete Two-Factor Authentication login system that strengthens conventional username/password authentication with a time-boxed, single-use, email-delivered OTP verification step. The system was built using Java 17 and Spring Boot 3.2.12 for the backend REST API, MySQL 8 for persistent, normalized storage of user, OTP, and audit data, and the Jakarta Mail API for reliable SMTP-based OTP delivery. Both a web application and a menu-driven console application were delivered, sharing the same underlying Service and DAO layers, demonstrating the value of a clean, layered architecture. All eight functional modules, registration, password-step login, OTP issuance, OTP verification, resend, change password, logout, and audit logging, were implemented, tested, and integrated into a coherent authentication workflow.

12.2 Why the System is Secure

The system's security rests on several complementary mechanisms working together rather than any single safeguard: BCrypt hashing ensures that neither passwords nor OTPs are ever stored or exposed in a directly usable plaintext form; the 5-minute expiry and single-use enforcement on OTPs sharply limit the window during which a captured or guessed code could be exploited; the 3-attempt limit defeats brute-force guessing of the six-digit OTP space; PreparedStatement-based queries throughout the DAO layer close off SQL injection as an attack vector; and comprehensive audit logging provides the visibility needed to detect and investigate suspicious authentication activity after the fact. Together, these measures raise the system's resistance to credential theft and account takeover substantially above what a conventional single-factor password system can offer.

12.3 Lessons Learned

DEPARTMENT OF COMPUTER APPLICATIONS

- Layered architecture (Controller-Service-DAO) significantly simplified both testing and reasoning about where specific responsibilities, and specific security controls, belonged in the codebase.
- Externalizing credentials into configuration files, rather than hardcoding them, is a small change with an outsized positive impact on the security hygiene of a project.
- Designing for failure modes, such as SMTP delivery delays or failures, expired OTPs, and exhausted attempts, is just as important as designing for the happy path, since a large fraction of real user interactions involve at least one retry or error state.
- Automated testing of security-critical logic (expiry, attempt limits, hashing correctness) is essential, since these are exactly the kinds of subtle correctness properties that are easy to get wrong and hard to notice through manual testing alone.
- Building both a web and a console interface around the same Service layer reinforced the practical value of separating business logic from any particular presentation layer.

DEPARTMENT OF COMPUTER APPLICATIONS

Chapter 13

References

1. Spring Boot Reference Documentation. Available: <https://docs.spring.io/spring-boot/documentation.html>
2. Spring Framework Documentation. Available: <https://docs.spring.io/spring-framework/reference/>
3. MySQL 8.0 Reference Manual. Oracle Corporation. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
4. Jakarta Mail Project Documentation. Eclipse Foundation. Available: <https://eclipse-ee4j.github.io/mail/>
5. jBCrypt Library. Available: <https://github.com/jeremyh/jBCrypt>
6. JUnit 5 User Guide. Available: <https://junit.org/junit5/docs/current/user-guide/>
7. Mockito Documentation. Available: <https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html>